

Trabajo Práctico Obligatorio: Sistema de Gestión Hotelera y Servicios para Huéspedes

Proceso de Desarrollo de software

Fase 2:Diseño orientado a objetos y patrones

PROFESOR: Lic Maria Angela Leon

INTEGRANTES:

- Juan Cruz borras, legajo: 1186835
- Victoria Davidson, legajo: 1181369
- Lola Cuartas, legajo: 1190036

Link de repositorio: <https://github.com/vdavidson1234/Sistema-de-Gestion-de-Hoteleria>

OBSERVACIONES:

Durante la revisión de la pantalla del frontend se realizaron distintas observaciones y correcciones sobre la interfaz. Algunas de estas modificaciones quedarán pendientes para ser desarrolladas en la Fase 3 del proyecto.

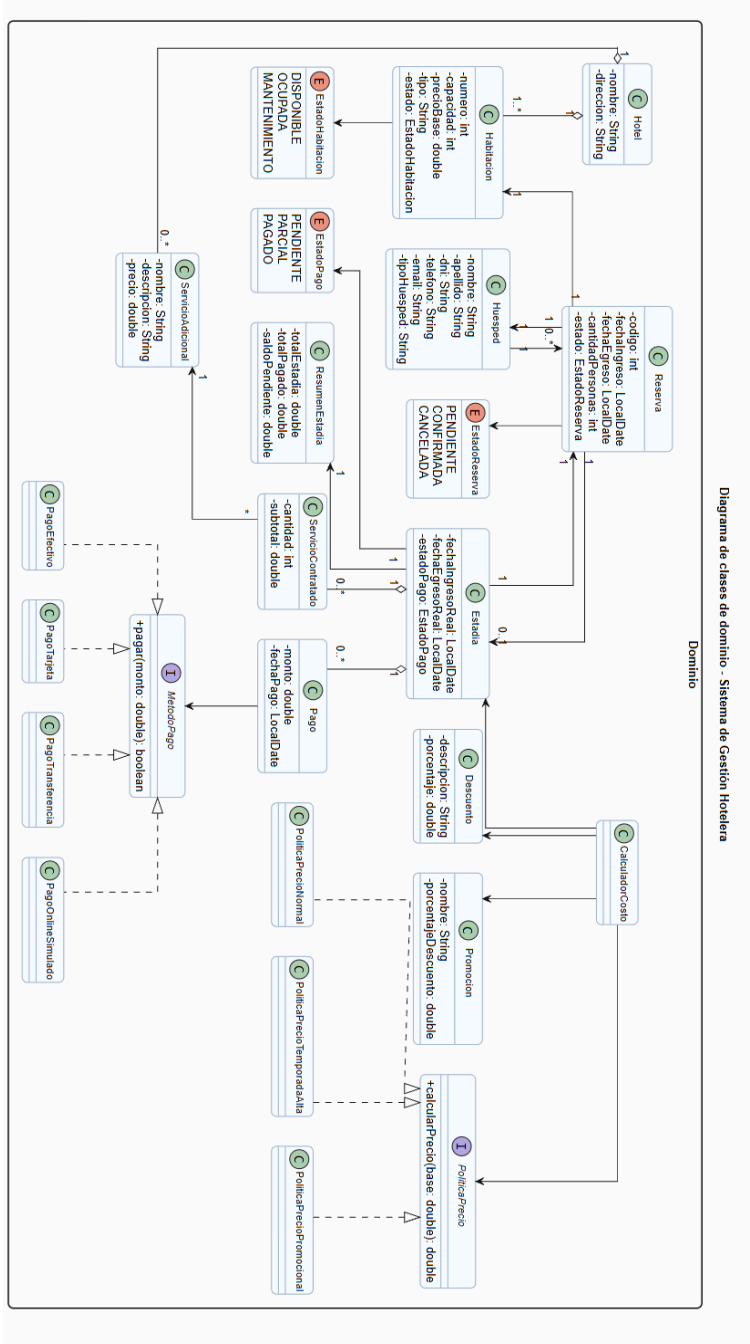
Entre los cambios propuestos, se estableció la necesidad de incorporar la opción de agregar acompañantes dentro de una reserva, junto con la posibilidad de seleccionar las habitaciones disponibles según la cantidad de personas. Además, se definió que la reserva deberá confirmarse únicamente una vez realizado el pago de una señal equivalente al 25% del valor total de la habitación.

También se planteó la necesidad de establecer precios fijos para los servicios adicionales, permitiendo calcular el monto final según las cantidades solicitadas y la cantidad de personas alojadas en una habitación. De esta manera, el sistema podrá reflejar de forma más precisa el costo total de la reserva.

Otra observación importante fue la validación de los datos de las personas incluidas en la reserva. Estos datos deberán guardarse en la base de datos para evitar solicitarlos nuevamente en futuras reservas, mejorando así la experiencia del usuario y reduciendo la carga repetitiva de información.

Por último, se indicó que el sistema deberá mostrar de forma clara el monto correspondiente al descuento aplicado, permitiendo que el usuario pueda visualizar el valor original, el descuento y el total final a pagar.

1.



El diagrama de clases UML representa la estructura principal del Sistema de Gestión Hotelera. En él se identifican las clases de dominio, sus atributos, las relaciones entre habitaciones, huéspedes, reservas y estadías, y también las interfaces necesarias para aplicar patrones de diseño como políticas de precio y métodos de pago. Este modelo sirve como base para la implementación inicial en Java.

2.

Diagrama de secuencia 1: Crear reserva

Este caso muestra cómo el recepcionista consulta disponibilidad, selecciona una habitación y crea una reserva para un huésped.

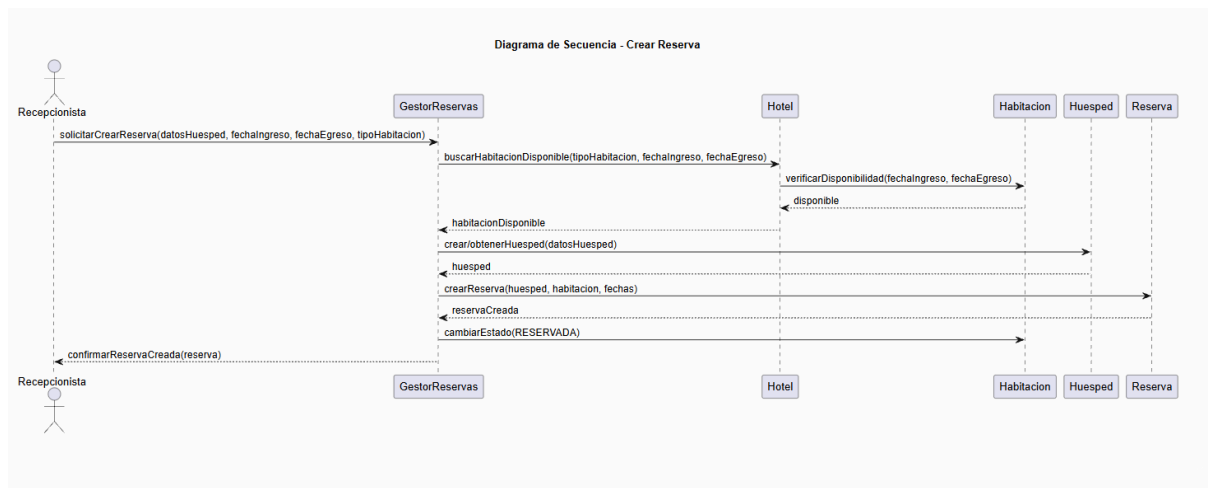
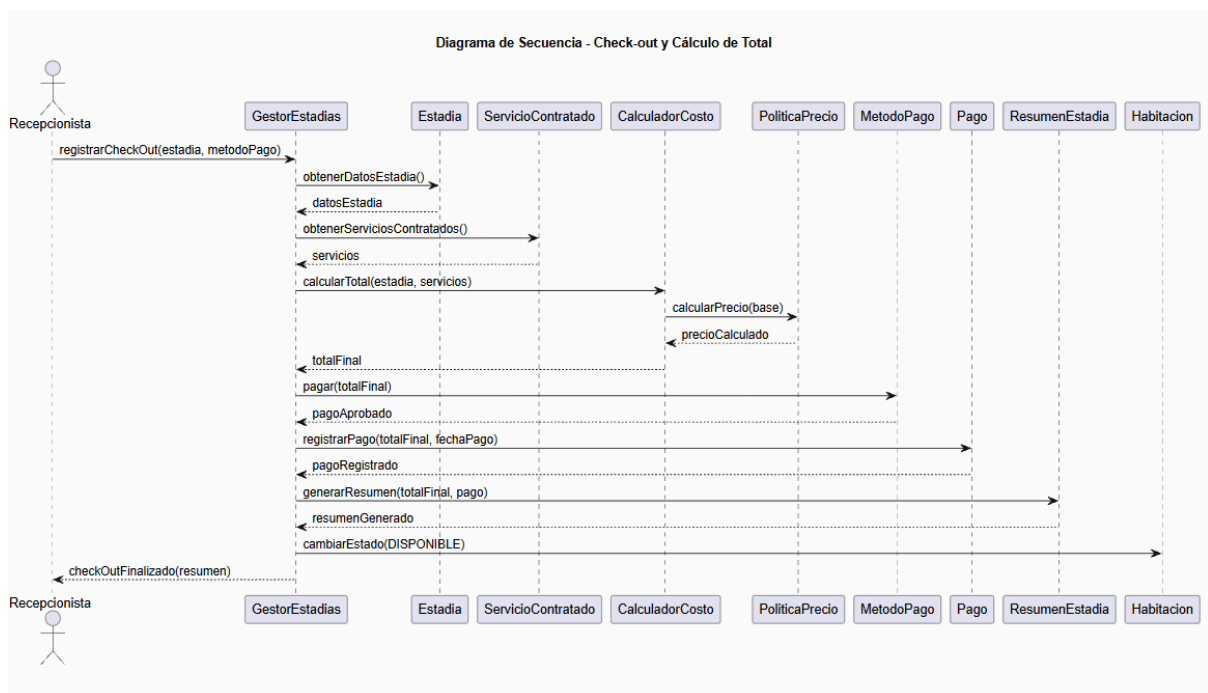


Diagrama de secuencia 2: Registrar check-out y calcular total

Este caso muestra cómo se cierra una estadía, se calculan servicios, descuentos/promociones y se registra el pago.



3. Identificación de patrones de diseño a aplicar

Para el Sistema de Gestión Hotelera se seleccionaron patrones de diseño que responden a necesidades concretas del sistema y que se encuentran reflejados tanto en el diagrama UML como en la primera versión del código Java.

Los patrones seleccionados son:

Patrón creacional: Factory Method / Simple Factory

Se aplica mediante la clase **HabitacionFactory**, ubicada en el paquete **gestionhotelera.factory**.

Este patrón se utiliza para centralizar la creación de objetos **Habitacion**, evitando que la lógica de construcción quede repetida en distintas partes del sistema.

Clases relacionadas:

- **HabitacionFactory**
- **Habitacion**
- **TipoHabitacion**
- **GestorHabitaciones**

Patrón estructural: Decorator

Se aplica en la gestión de servicios adicionales de una estadía, mediante el paquete **gestionhotelera.decorator**.

Este patrón permite agregar servicios extra, como desayuno, spa, cochera o lavandería, sin modificar directamente la clase principal de estadía.

Clases relacionadas:

- **ServicioEstadia**
- **ServicioBase**
- **ServicioDecorator**
- **DesayunoDecorator**
- **SpaDecorator**
- **CocheraDecorator**
- **LavanderiaDecorator**
- **Estadia**

Patrón de comportamiento: Strategy para políticas de precio y descuentos

Se aplica en el cálculo del costo final de una estadía, mediante el paquete **gestionhotelera.strategy**.

Este patrón permite cambiar la forma de calcular precios y descuentos sin modificar la clase **CalculadorCosto**.

Clases relacionadas:

- **PoliticaPrecio**
- **PoliticaPrecioNormal**
- **PoliticaPrecioTemporadaAlta**
- **PoliticaPrecioPromocional**
- **DescuentoStrategy**
- **DescuentoClienteFrecuente**
- **DescuentoTemporadaBaja**
- **DescuentoPromocionEspecial**
- **CalculadorCosto**

Patrón de comportamiento: Strategy para métodos de pago

También se aplica Strategy en los métodos de pago, mediante el paquete **gestionhotelera.pagos**.

El sistema permite procesar pagos usando distintas formas de cobro, como efectivo, tarjeta, transferencia o pago online simulado.

Clases relacionadas:

- **MetodoPago**
- **PagoEfectivo**
- **PagoTarjeta**
- **PagoTransferencia**
- **PagoOnlineSimulado**
- **Pago**
- **GestorPagos**

Patrón de comportamiento: State

Se aplica en el ciclo de vida de una reserva, mediante el paquete **gestionhotelera.state**.

Este patrón permite que una reserva cambie su comportamiento según su estado actual: pendiente, confirmada, cancelada o finalizada.

Clases relacionadas:

- **EstadoReservaComportamiento**

- **ReservaPendienteState**
- **ReservaConfirmadaState**
- **ReservaCanceladaState**
- **ReservaFinalizadaState**
- **Reserva**

Patron de comportamiento: Adapter

Las clases utilizadas para recibir y guardar los datos de la base de datos pueden considerarse adapters, ya que cumplen la función de adaptar la información entre dos representaciones distintas: por un lado, los objetos del dominio del sistema, como Hotel, Habitación, Reserva y Huesped; y por otro lado, las tablas y registros de la base de datos SQL.

En particular, la clase HotelPersistence funciona como un adapter de persistencia porque se encarga de convertir los objetos del sistema en instrucciones SQL para guardarlos en la base de datos, y también realiza el proceso inverso: lee los datos desde la base de datos y reconstruye los objetos correspondientes del dominio.

Además, la clase Database encapsula los detalles técnicos de conexión con SQLite y la creación del esquema de tablas, evitando que el resto del sistema tenga que conocer directamente cómo se realiza la conexión o cómo se inicializa la base de datos.

Por este motivo, estas clases actúan como una capa intermedia entre la lógica del sistema y la base de datos. Si bien también podrían describirse como clases de tipo Repository o DAO, cumplen claramente un rol de Adapter, ya que aíslan al dominio de los detalles técnicos de persistencia y traducen entre el modelo orientado a objetos y el modelo relacional de la base de datos.

4. Justificación de cada patrón seleccionado

Factory Method / Simple Factory

El patrón Factory Method / Simple Factory se utiliza para resolver la creación de habitaciones dentro del sistema. En lugar de crear habitaciones manualmente desde distintas partes del código, se centraliza esa responsabilidad en la clase **HabitacionFactory**.

Esto permite que el sistema sea más ordenado y fácil de mantener. Si en el futuro el hotel incorpora nuevos tipos de habitaciones, como habitación premium, familiar o presidencial, la lógica de creación puede modificarse o ampliarse desde la fábrica, sin afectar directamente a los controladores ni a las demás clases del dominio.

En el código, **GestorHabitaciones** utiliza **HabitacionFactory** para crear una habitación y luego registrarla dentro del hotel. De esta manera, el controlador no se encarga de construir el objeto manualmente, sino que delega esa responsabilidad a la fábrica.

Decorator

El patrón Decorator se utiliza para agregar servicios adicionales a una estadía de forma flexible. En un hotel, una estadía puede tener distintos servicios extra, como desayuno, spa, cochera o lavandería. Estos servicios pueden combinarse de diferentes maneras según lo que consuma cada huésped.

En lugar de crear una clase diferente para cada combinación posible de servicios, se utiliza una estructura de decoradores. La clase **ServicioBase** representa el servicio inicial, mientras que clases como **DesayunoDecorator**, **SpaDecorator**, **CocheraDecorator** y **LavanderiaDecorator** agregan información y costo adicional.

Este patrón permite extender el comportamiento de los servicios sin modificar la clase base. También evita que el sistema tenga una estructura rígida o llena de condicionales para calcular cada servicio extra.

Strategy para políticas de precio y descuentos

El patrón Strategy se utiliza en el cálculo de precios y descuentos porque el costo final de una estadía puede variar según distintas políticas comerciales. Por ejemplo, el hotel puede aplicar precio normal, precio de temporada alta o precio promocional. También puede aplicar descuentos por cliente frecuente, temporada baja o promociones especiales.

En el código, la interfaz **PoliticaPrecio** define el contrato para las políticas de precio. Sus implementaciones son **PoliticaPrecioNormal**, **PoliticaPrecioTemporadaAlta** y **PoliticaPrecioPromocional**. Por otro lado, la interfaz **DescuentoStrategy** define el comportamiento de los descuentos, con implementaciones como **DescuentoClienteFrecuente**, **DescuentoTemporadaBaja** y **DescuentoPromocionEspecial**.

La clase **CalculadorCosto** recibe una política de precio y una estrategia de descuento, y calcula el total sin depender de clases concretas. Esto permite cambiar la forma de calcular el precio sin modificar el calculador.

Strategy para métodos de pago

El patrón Strategy también se aplica en los métodos de pago. El sistema permite utilizar diferentes formas de pago, como efectivo, tarjeta, transferencia o pago online simulado.

Todas estas formas de pago implementan la interfaz **MetodoPago**, que define el método pagar. Las clases **PagoEfectivo**, **PagoTarjeta**, **PagoTransferencia** y **PagoOnlineSimulado** resuelven el pago de manera independiente.

Esto permite que la clase Pago no dependa de un único método de pago. Si en el futuro se desea agregar otro medio, como billetera virtual o Mercado Pago, se puede crear una nueva clase que implemente **MetodoPago**, sin modificar la lógica principal del sistema.

State

El patrón State se utiliza para manejar el ciclo de vida de una reserva. Una reserva no siempre puede realizar las mismas acciones. Por ejemplo, una reserva pendiente puede confirmarse o cancelarse, una reserva confirmada puede finalizarse, y una reserva cancelada no debería poder confirmarse nuevamente.

Para resolver esto, se utiliza la interfaz **EstadoReservaComportamiento** y las clases concretas **ReservaPendienteState**, **ReservaConfirmadaState**, **ReservaCanceladaState** y **ReservaFinalizadaState**.

La clase Reserva delega las acciones **confirmar**, **cancelar** y **finalizar** al estado actual. Esto evita tener muchos condicionales dentro de la clase Reserva y permite que cada estado tenga su propia lógica.

Adapter

Se considera que las clases de persistencia funcionan como adapters porque permiten que el sistema trabaje con objetos propios del dominio sin depender directamente de la estructura interna de la base de datos. Estas clases se encargan de traducir la información entre el modelo orientado a objetos y el modelo relacional utilizado por SQL.

Gracias a esta separación, el resto del sistema no necesita conocer detalles técnicos como consultas SQL, conexiones, tablas o columnas. Esto mejora la organización del código, reduce el acoplamiento y facilita posibles cambios futuros, por ejemplo, reemplazar SQL por otra base de datos sin modificar la lógica principal del sistema.

5. Identificación de principios SOLID aplicados

En el Sistema de Gestión Hotelera se aplican distintos principios SOLID con el objetivo de lograr un diseño orientado a objetos más claro, mantenible, flexible y fácil de extender.

SRP - Single Responsibility Principle

El principio de responsabilidad única establece que cada clase debe tener una única responsabilidad o motivo de cambio. En el proyecto, este principio se aplica separando las tareas del sistema en distintas clases, evitando que una sola clase concentre demasiadas funciones.

Por ejemplo, la clase **Huesped** se encarga únicamente de representar los datos del huésped, como nombre, apellido, DNI, teléfono y email. La clase **Habitacion** representa la información de una habitación, incluyendo su número, tipo, capacidad, precio base y estado. La clase **Reserva** representa una reserva realizada por un huésped para una habitación determinada, incluyendo fechas y estado de reserva.

También se aplica este principio en las clases de control. **GestorReservas** se encarga de administrar la creación, confirmación y cancelación de reservas. **GestorHabitaciones** se

encarga de registrar habitaciones y consultar disponibilidad. **GestorEstadias** administra el ingreso, egreso, servicios consumidos y finalización de estadías. Por su parte, **CalculadorCosto** tiene como responsabilidad calcular el costo final de una estadía, sin encargarse de registrar huéspedes, crear reservas o procesar pagos.

De esta manera, cada clase tiene una función clara dentro del sistema, lo que facilita la comprensión del código, su mantenimiento y futuras modificaciones.

OCP - Open/Closed Principle

El principio abierto/cerrado establece que las clases deben estar abiertas a la extensión, pero cerradas a la modificación. Esto significa que el sistema debe permitir agregar nuevas funcionalidades sin tener que modificar constantemente el código ya existente.

En el proyecto, este principio se aplica principalmente en las políticas de precio, descuentos, métodos de pago y servicios adicionales.

En el caso de las políticas de precio, el sistema utiliza la interfaz **PoliticaPrecio**, con implementaciones como **PoliticaPrecioNormal**, **PoliticaPrecioTemporadaAlta** y **PoliticaPrecioPromocional**. Si en el futuro el hotel quisiera agregar una nueva política, como una política de precio para fines de semana o feriados, se podría crear una nueva clase que implemente **PoliticaPrecio**, sin modificar la clase **CalculadorCosto**.

Lo mismo ocurre con los descuentos. La interfaz **DescuentoStrategy** permite agregar nuevos descuentos, como descuento por cliente frecuente, descuento por temporada baja o promoción especial. Si se necesitara agregar un nuevo descuento, se podría crear una nueva clase sin alterar el funcionamiento general del sistema.

También se aplica OCP en los métodos de pago. Todas las formas de pago implementan la interfaz **MetodoPago**, como **PagoEfectivo**, **PagoTarjeta**, **PagoTransferencia** y **PagoOnlineSimulado**. Si en el futuro se quisiera agregar un nuevo medio de pago, como billetera virtual, solo sería necesario crear una nueva clase que implemente esa interfaz.

Además, este principio se ve en el uso del patrón Decorator para los servicios adicionales. Si se desea agregar un nuevo servicio, como cena, excursión o servicio de habitación, se puede crear un nuevo decorador sin modificar la estructura ya existente.

LSP - Liskov Substitution Principle

El principio de sustitución de Liskov establece que una clase hija o una implementación debe poder reemplazar a su clase padre o interfaz sin alterar el funcionamiento correcto del sistema.

En el proyecto, este principio se aplica principalmente en las interfaces utilizadas para estrategias, métodos de pago, servicios adicionales y estados de reserva.

Por ejemplo, cualquier clase que implemente **MetodoPago**, como **PagoEfectivo**, **PagoTarjeta**, **PagoTransferencia** o **PagoOnlineSimulado**, puede utilizarse en cualquier parte del sistema donde se espere un objeto de tipo **MetodoPago**. Todas estas clases respetan el mismo contrato y permiten ejecutar el método **pagar**.

También se aplica en las políticas de precio. Las clases **PoliticaPrecioNormal**, **PoliticaPrecioTemporadaAlta** y **PoliticaPrecioPromocional** pueden reemplazarse entre sí porque todas implementan la interfaz **PoliticaPrecio** y cumplen con el método esperado para calcular el precio.

Lo mismo ocurre con los descuentos. Las clases **DescuentoClienteFrecuente**, **DescuentoTemporadaBaja** y **DescuentoPromocionEspecial** implementan la interfaz **DescuentoStrategy**, por lo tanto pueden sustituirse sin afectar el funcionamiento de **CalculadorCosto**.

Este principio también aparece en el patrón State aplicado a las reservas. Las clases **ReservaPendienteState**, **ReservaConfirmadaState**, **ReservaCanceladaState** y **ReservaFinalizadaState** implementan el comportamiento definido por **EstadoReservaComportamiento**. La clase **Reserva** puede trabajar con cualquiera de esos estados sin depender directamente de una implementación concreta.

Gracias a este principio, el sistema puede cambiar políticas de precio, descuentos, métodos de pago o estados sin romper la lógica principal del programa.

DIP - Dependency Inversion Principle

El principio de inversión de dependencias establece que las clases de alto nivel no deben depender de clases concretas, sino de abstracciones. Esto permite reducir el acoplamiento y hacer que el sistema sea más flexible.

En el Sistema de Gestión Hotelera, este principio se aplica cuando clases importantes dependen de interfaces y no de implementaciones específicas.

Por ejemplo, la clase **CalculadorCosto** no depende directamente de clases concretas como **PoliticaPrecioNormal**, **PoliticaPrecioTemporadaAlta** o **DescuentoClienteFrecuente**. En cambio, trabaja con las interfaces **PoliticaPrecio** y **DescuentoStrategy**. Esto permite que el cálculo del costo final pueda cambiar según la política de precio o el descuento utilizado, sin modificar la clase calculadora.

También se aplica en los métodos de pago. La clase **Pago** o el gestor encargado de registrar pagos trabaja con la interfaz **MetodoPago**, y no con una clase concreta como **PagoTarjeta** o **PagoEfectivo**. Esto permite cambiar el método de pago utilizado sin afectar el resto del sistema.

Además, el uso de interfaces como **ServicioEstadia** permite que los servicios adicionales se manejen de forma flexible mediante decoradores, sin que la estadía dependa directamente de cada servicio específico.

Este principio ayuda a que el sistema sea más fácil de ampliar, probar y mantener, ya que las clases principales no quedan fuertemente acopladas a implementaciones concretas.

En conclusión, la aplicación de los principios SOLID permite que el Sistema de Gestión Hotelera tenga una estructura más clara y flexible. SRP permite separar responsabilidades, OCP facilita la incorporación de nuevas funcionalidades, LSP asegura que las implementaciones puedan sustituirse correctamente y DIP reduce la dependencia entre clases concretas. Estos principios contribuyen a que el proyecto sea más mantenible y coherente con un diseño orientado a objetos.

6. Identificación de patrones GRASP aplicados

Controller

El patrón GRASP Controller se aplica en las clases que coordinan las operaciones principales del sistema.

Clases donde se aplica:

- **GestorReservas**
- **GestorHabitaciones**
- **GestorEstadias**
- **GestorPagos**

Estas clases actúan como intermediarias entre la ejecución del programa y las clases de dominio. Por ejemplo, **GestorReservas** recibe la solicitud de crear una reserva, valida disponibilidad, crea la reserva y la registra en el hotel. De esta forma, la lógica de coordinación no queda distribuida de manera desordenada entre las entidades.

Creator

El patrón GRASP Creator se aplica asignando la responsabilidad de crear objetos a las clases que tienen la información necesaria para hacerlo.

Ejemplos dentro del sistema:

- **GestorReservas** crea objetos **Reserva**, porque conoce el huésped, la habitación elegida, la fecha de ingreso y la fecha de egreso.
- **GestorEstadias** crea objetos **Estadia**, porque recibe una reserva confirmada y las fechas reales de ingreso y egreso.
- **GestorPagos** crea objetos **Pago**, porque recibe la estadía, el monto y el método de pago.
- **HabitacionFactory** crea objetos **Habitacion**, porque centraliza la lógica de construcción de habitaciones.

Esto permite que la creación de objetos quede ubicada en clases coherentes con el problema.

Information Expert

El patrón GRASP Information Expert se aplica asignando responsabilidades a las clases que poseen la información necesaria para resolverlas.

Ejemplos dentro del sistema:

- **Hotel** consulta disponibilidad porque conoce sus habitaciones y reservas.
- **Habitacion** sabe si está disponible porque conoce su propio estado.
- **Reserva** calcula la cantidad de noches y sabe si bloquea disponibilidad porque conoce sus fechas y estado.
- **Estadia** calcula el total de servicios, total pagado y saldo pendiente porque conoce sus servicios y pagos.
- **Pago** procesa el pago porque contiene el monto y el método de pago elegido.

Este patrón permite que cada responsabilidad quede en la clase que tiene la información más adecuada para realizarla.

Low Coupling

El patrón GRASP Low Coupling se aplica buscando reducir la dependencia directa entre clases.

Ejemplos dentro del sistema:

- **CalculadorCosto** no depende de descuentos concretos, sino de **DescuentoStrategy**.
- **Pago** no depende de pagos concretos, sino de **MetodoPago**.
- Los servicios adicionales se manejan mediante **ServicioEstadia**, permitiendo usar decoradores sin acoplar la estadía a servicios específicos.

Esto permite modificar o agregar funcionalidades con menor impacto sobre el resto del sistema.

High Cohesion

El patrón GRASP High Cohesion se aplica manteniendo clases con responsabilidades relacionadas y bien definidas.

Ejemplos dentro del sistema:

- El paquete **dominio** contiene las entidades principales del negocio.
- El paquete **control** contiene clases coordinadoras del sistema.
- El paquete **factory** contiene la fábrica de habitaciones.
- El paquete **decorator** contiene los servicios adicionales.
- El paquete **strategy** contiene las políticas de precio y descuentos.

- El paquete **pagos** contiene los métodos de pago.
- El paquete **state** contiene los estados de reserva.

7. Estructura inicial de paquetes del proyecto Java

La primera versión del proyecto Java se encuentra organizada en paquetes, separando responsabilidades según el tipo de clase.

La estructura inicial es la siguiente:

```

src/
├── gestionhotelera/
│   └── App.java
└── control/
    ├── CalculadorCosto.java
    ├── GestorEstadias.java
    ├── GestorHabitaciones.java
    ├── GestorPagos.java
    ├── GestorReservas.java
    ├── NotificadorReserva.java
    └── ValidadorDisponibilidad.java

└── dominio/
    ├── Estadia.java
    ├── EstadoHabitacion.java
    ├── EstadoPago.java
    ├── EstadoReserva.java
    ├── Habitacion.java
    ├── Hotel.java
    ├── Huesped.java
    ├── Pago.java
    ├── Reserva.java
    ├── ServicioConsumido.java
    ├── ServicioEstadia.java
    ├── StaySummary.java
    └── TipoHabitacion.java

└── factory/
    └── HabitacionFactory.java

└── decorator/
    ├── CocheraDecorator.java
    ├── DesayunoDecorator.java
    ├── LavanderiaDecorator.java
    └── ServicioBase.java
    
```

- ServicioDecorator.java
- SpaDecorator.java
- strategy/
 - DescuentoClienteFrecuente.java
 - DescuentoPromocionEspecial.java
 - DescuentoStrategy.java
 - DescuentoTemporadaBaja.java
 - PoliticaPrecio.java
 - PoliticaPrecioNormal.java
 - PoliticaPrecioPromocional.java
 - PoliticaPrecioTemporadaAlta.java
- pagos/
 - MetodoPago.java
 - PagoEfectivo.java
 - PagoOnlineSimulado.java
 - PagoTarjeta.java
 - PagoTransferencia.java
- state/
 - EstadoReservaComportamiento.java
 - ReservaCanceladaState.java
 - ReservaConfirmadaState.java
 - ReservaFinalizadaState.java
 - ReservaPendienteState.java
- persistence/
 - Database.java
 - HotelPersistence.java
- ui/
 - HotelGUI.java

Esta estructura permite separar correctamente las clases de dominio, los controladores, los patrones de diseño, la persistencia y la interfaz gráfica.

8. Primera versión del código con clases principales, interfaces y relaciones

La primera versión del código ya contiene las clases principales del sistema y refleja las relaciones planteadas en el diagrama UML.

Clases de dominio implementadas

El paquete **gestionhotelera.dominio** contiene las clases centrales del problema:

- **Hotel**: representa al hotel y agrupa habitaciones y reservas.
- **Habitacion**: representa una habitación con número, capacidad, precio base, tipo y estado.
- **Huesped**: representa los datos del huésped.
- **Reserva**: representa la reserva de una habitación, asociada a un huésped, una habitación y fechas.
- **Estadia**: representa la permanencia real del huésped, incluyendo servicios consumidos y pagos.
- **Pago**: representa un pago realizado durante la estadía.
- **StaySummary**: resume una estadía para mostrar información de forma simple.
- **ServicioEstadia y ServicioConsumido**: permiten representar servicios adicionales.
- **EstadoHabitacion, EstadoPago, EstadoReserva y TipoHabitacion**: enumeraciones utilizadas para representar estados y tipos.

Controladores implementados

El paquete **gestionhotelera.control** contiene clases que coordinan las operaciones principales:

- **GestorHabitaciones**: crea y registra habitaciones usando **HabitacionFactory**.
- **GestorReservas**: crea, confirma y cancela reservas.
- **GestorEstadias**: registra estadías, agrega servicios, calcula costos y finaliza estadías.
- **GestorPagos**: registra pagos en una estadía.
- **CalculadorCosto**: calcula el costo final combinando política de precio y descuento.
- **ValidadorDisponibilidad**: consulta habitaciones disponibles.
- **NotificadorReserva**: notifica confirmaciones o cancelaciones de reservas.

Interfaces implementadas

El sistema incluye interfaces para aplicar polimorfismo y desacoplamiento:

- **PoliticaPrecio**: define cómo se calcula una política de precio.
- **DescuentoStrategy**: define cómo se aplica un descuento.
- **MetodoPago**: define cómo se procesa un método de pago.
- **ServicioEstadia**: define los datos comunes de un servicio agregado a una estadía.
- **ServicioConsumido**: representa un servicio consumido dentro de una estadía.
- **EstadoReservaComportamiento**: define el comportamiento de los estados de una reserva.

Relaciones principales implementadas

Las relaciones principales del sistema son:

- Un **Hotel** contiene muchas **Habitacion** y muchas **Reserva**.

- Una **Reserva** se asocia a un **Huesped** y a una **Habitacion**.
- Una **Estadia** se crea a partir de una **Reserva** confirmada.
- Una **Estadia** puede tener varios servicios consumidos y varios pagos.
- Un Pago utiliza un **MetodoPago**.
- **CalculadorCosto** utiliza **PoliticaPrecio** y **DescuentoStrategy**.
- Reserva utiliza un objeto de tipo **EstadoReservaComportamiento** para manejar su estado interno.
- **GestorReservas**, **GestorHabitaciones**, **GestorEstadias** y **GestorPagos** coordinan las operaciones del sistema.

Demostración funcional inicial

La clase App funciona como punto de entrada del sistema. En la demo se crea un hotel, se registran habitaciones, se crea un huésped, se genera una reserva, se confirma, se registra una estadía, se agregan servicios adicionales, se calcula el costo final, se registran pagos y se finaliza la estadía.

Este flujo permite demostrar el funcionamiento básico del sistema y la interacción entre las clases principales.

Además, el proyecto cuenta con una interfaz gráfica en el paquete **gestionhotelera.ui**, mediante la clase **HotelGUI**, lo que permite avanzar hacia una demostración más visual del sistema.

9. Informe breve de avance individual por integrante

Victoria Davidson:

Durante esta fase se avanzó en el diseño orientado a objetos del Sistema de Gestión Hotelera, tomando como base el diagrama de clases UML realizado previamente. Se definieron las responsabilidades principales de cada clase y se seleccionaron los patrones de diseño que serán aplicados en la solución.

También se avanzó en la implementación inicial del proyecto en Java utilizando Visual Studio Code. Se organizaron los paquetes del sistema, separando clases de dominio, controladores, patrones de diseño, métodos de pago, estados, persistencia e interfaz gráfica.

En cuanto al código, se implementaron las clases principales del dominio, como **Hotel**, **Habitacion**, **Huesped**, **Reserva**, **Estadia** y **Pago**. También se implementaron controladores como **GestorReservas**, **GestorHabitaciones**, **GestorEstadias** y **GestorPagos**, junto con interfaces y clases relacionadas con patrones de diseño.

Se incorporaron los patrones Factory Method / Simple Factory, Decorator, Strategy y State. Además, se aplicaron principios SOLID como SRP, OCP y DIP, y patrones GRASP como Controller, Creator, Information Expert, Low Coupling y High Cohesion.

La primera versión del código ya permite ejecutar un flujo funcional básico: crear habitaciones, registrar un huésped, crear y confirmar una reserva, registrar una estadía, agregar servicios adicionales, calcular el costo final, registrar pagos y finalizar la estadía.

Este avance deja preparada la base del sistema para continuar con la Fase 3, donde se completará la implementación, se realizarán pruebas, se actualizarán los diagramas UML y se preparará la defensa final del proyecto.

Juan Cruz Borrás:

Durante esta fase participé en el avance del diseño orientado a objetos del Sistema de Gestión Hotelera, colaborando en la revisión del diagrama de clases UML y en la definición de responsabilidades de las clases principales. Se trabajó sobre la relación entre entidades como **Hotel**, **Habitacion**, **Reserva**, **Huesped**, **Estadia**, **Pago** y los distintos gestores del sistema.

También colaboré en la organización inicial del proyecto Java en Visual Studio Code, teniendo en cuenta la separación de paquetes según su función: dominio, controladores, patrones de diseño, pagos, estados, persistencia e interfaz gráfica. Esta organización permite que el código sea más claro, mantenible y coherente con el diseño planteado.

Además, participé en la identificación de patrones de diseño aplicados al sistema, como Factory Method / Simple Factory, Decorator, Strategy y State. También se revisó la aplicación de principios SOLID y patrones GRASP, especialmente en la asignación de responsabilidades, el uso de interfaces y la reducción del acoplamiento entre clases.

Por ultimo, trabaje principalmente en el diseño y creación de la base de datos. Definimos una base de datos SQL relacional, ya que el sistema requiere persistencia e integridad fuertes y SQL es perfecto debido a que garantiza propiedades ACID completas. La base estará levantada en docker como contenedor para que no funcione de manera local en cada dispositivo. La correcta identificación de las entidades del dominó permitió realizar el diagrama entidad-relación con mayor precisión y facilidad. Los próximos pasos son conectar efectivamente la base de datos a la aplicación mediante las clases que funcionan como adapters.

Lola Cuartas:

Durante esta fase colaboré en la revisión general del diseño del Sistema de Gestión Hotelera, tomando como referencia el diagrama de clases UML y los requerimientos definidos para el proyecto. Participé en la identificación de las clases principales del sistema y en la organización de sus responsabilidades dentro del modelo orientado a objetos.

También fui partícipe de el armado de la estructura inicial del proyecto Java, verificando que las clases de dominio, controladores y paquetes estuvieran organizados de forma clara. Además, contribuí en la revisión de los patrones de diseño seleccionados y en la relación entre el diseño UML y la primera versión del código.

Mi aporte durante esta etapa permitió avanzar en la Fase 2 y en la preparación de la base del sistema para continuar con la implementación completa en la siguiente fase.